Jetpack Compose aplicado a Scrapp: diseño e implementación de la interfaz Android para el control de mermas

Víctor Emmanuel Velasco Martínez (VM251307)

Fernando José Gómez Hernández (GH251230)

José Eduardo Aquino Medrano (AM252078)

William Eduardo Montano Aguilar (MA251192)

Grupo CTRL+CAFE

Dirección de Educación a Distancia, Universidad Don Bosco

DSM941 G01T: Desarrollo de Software para Móviles

Alexander Alberto Siguenza Campos

21 de octubre de 2026

Proyecto de cátedra. Etapa 3, Avance 1: Foro de Discusión 1

Índice

Introducción 6

Fundamentos de Jetpack Compose 7

Del sistema de vistas XML al paradigma declarativo 7

Funciones composables y recomposición 8

Estado, remember y elevación de estado 9

Modifier, diseño y listas diferidas 10

Efectos secundarios 10

Material Design 3 y temas 11

Navigation Compose 11

Arquitectura de Scrapp con Jetpack Compose 11

De MVC con layouts XML a MVC con funciones composables 11

Capa de datos compartida 14

Dependencias del proyecto 14

Sistema de diseño de Scrapp 15

Tema Material 3 a partir del prototipo 15

Componentes reutilizables 17

Jetpack Compose aplicado a cada pantalla 18

Inicio de sesión y registro 18

Dashboard de producción 21

Monitor de lotes 22

Ingreso de lotes 24

Registro de mermas 25

Panel de gerencia 27

Navegación 29

Grafo de navegación 29

Barra inferior por rol y protección de rutas 30

Validación de formularios 32

Análisis propio: ventajas y limitaciones 33

Ventajas observadas durante el desarrollo 33

Limitaciones y mejoras para el Avance 2 34

Conclusiones 35

Referencias 37

Índice de figuras

Figura 1 *Flujo de datos unidireccional en el inicio de sesión de Scrapp* 13

Figura 2 *Prototipo de las pantallas principales de Scrapp elaborado en Figma durante la Etapa 1* 16

Figura 3 *Sobrecarga de ChipSemaforo para el estado de caducidad de un lote* 18

Figura 4 *Pantalla de inicio de sesión con validaciones activas* 19

Figura 5 *Pantalla de registro de usuario* 20

Figura 6 *Dashboard de producción del rol Operativo* 22

Figura 7 *Monitor de lotes con el semáforo de caducidad* 23

Figura 8 *Recálculo del semáforo cada vez que el monitor vuelve a primer plano* 24

Figura 9 *Formulario de ingreso de lote con cálculo previo de caducidad* 25

Figura 10 *Formulario de registro de mermas* 27

Figura 11 *Panel de gerencia del rol Administrador* 28

Figura 12 *Grafo de navegación implementado en ScrappNavHost* 29

Figura 13 *Protección del panel de gerencia según el rol del usuario* 31

Índice de tablas

Tabla 1 *Comparación entre el sistema de vistas XML y Jetpack Compose* 7

Tabla 2 *Correspondencia de las capas MVC entre la Etapa 2 y la versión Android* 12

Tabla 3 *Dependencias de interfaz y arquitectura del proyecto scrapp-android* 14

Tabla 4 *Componentes composables reutilizables de Scrapp* 17

Tabla 5 *Destinos del grafo de navegación de Scrapp* 30

Tabla 6 *Validaciones implementadas en los formularios de Scrapp* 32

Jetpack Compose aplicado a Scrapp: diseño e implementación de la interfaz Android para el control de mermas

Introducción

Scrapp es la aplicación que propusimos como grupo CTRL+CAFE en la Etapa 1 para llevar el control de las mermas de un restaurante, es decir, de los productos que se pierden por vencimiento, deterioro, sobreproducción, errores de preparación o porque sobraron al cierre. En la Etapa 2 programamos en Kotlin, en modo consola, el núcleo de la lógica: el semáforo de caducidad de lotes, la proyección de producción diaria y el cálculo financiero de pérdidas. En la Etapa 3 esa lógica pasa a una aplicación Android, y para el primer avance la interfaz tenía que hacerse con Jetpack Compose.

En este documento explicamos qué es Jetpack Compose y qué partes de él usamos en Scrapp, con las razones de cada decisión. Por eso la teoría va acotada: tratamos cada concepto solo hasta donde explica algo del proyecto, como la forma de una pantalla, el manejo del estado o la navegación. Primero resumimos los fundamentos a partir de la documentación oficial de Android Developers. Después describimos la arquitectura, el sistema de diseño, las pantallas, la navegación y las validaciones. Al final dejamos nuestro análisis de lo que funcionó, lo que no y lo que vamos a cambiar en el Avance 2.

Hay un cambio respecto a la Etapa 1 que conviene aclarar desde ya. En la propuesta original describimos la capa de vista con layouts en XML (CTRL+CAFE, 2026). Con Compose la vista ya no es un archivo de marcado que se infla y luego se modifica desde código. Ahora son funciones de Kotlin que dibujan la interfaz a partir del estado. Mantuvimos la arquitectura Modelo-Vista-Controlador, aunque el controlador cambió de forma, y más adelante explicamos por qué.

Fundamentos de Jetpack Compose

Del sistema de vistas XML al paradigma declarativo

Jetpack Compose es el kit de herramientas moderno y declarativo de Android para construir interfaces nativas (Android Developers, s. f.-f). Google publicó la versión estable 1.0 en julio de 2021 y lo presentó como una forma de construir aplicaciones con mucho menos código gracias a APIs intuitivas de Kotlin (Bellini y Butcher, 2021). Lo que más cambia frente al sistema de vistas tradicional es la manera de pensar la interfaz; la Tabla 1 resume las diferencias.

Con el sistema de vistas, la interfaz se declara en XML y después se modifica a mano: hay que buscar cada vista por su identificador y cambiarle el texto, el color o la visibilidad cada vez que cambian los datos. Mantener sincronizado lo que se ve con los datos queda a cargo del programador. En Compose la interfaz se describe como una función del estado. Conceptualmente, la pantalla se regenera desde cero y solo se aplican los cambios necesarios (Android Developers, s. f.-f). Si el estado cambia, Compose vuelve a ejecutar las funciones afectadas y actualiza únicamente las partes que dependen de él.

Tabla 1

Comparación entre el sistema de vistas XML y Jetpack Compose

Aspecto	Sistema de vistas (XML)	Jetpack Compose	Consecuencia para Scrapp
Definición de la interfaz	Archivos XML de layout inflados desde una Activity o Fragment	Funciones de Kotlin anotadas con `@Composable`	Toda la vista vive en el paquete `vista/`, en el mismo lenguaje que el modelo y los controladores
Actualización de la pantalla	Imperativa: `findViewById` y llamadas como `setText` o `setVisibility`	Declarativa: la pantalla se recompone cuando cambia el estado que lee	El chip del semáforo cambia de color con solo actualizar el estado del lote; no hay código que lo pinte a mano
Listas	`RecyclerView` con `Adapter` y `ViewHolder`	`LazyColumn` con `items` y una clave por elemento	El monitor de lotes y el panel de gerencia se resuelven en pocas líneas

Aspecto	Sistema de vistas (XML)	Jetpack Compose	Consecuencia para Scrapp
Reutilización	Estilos, `include` y vistas personalizadas	Funciones composables con parámetros	Ocho componentes propios en `vista/componentes/` usados por varias pantallas
Temas	Temas y estilos en recursos XML	`MaterialTheme` con `ColorScheme` y tipografía en Kotlin	La paleta del prototipo se define una sola vez en `ui/theme/`
Navegación	Fragments y grafo XML de Navigation	`NavHost` y destinos composables	El grafo completo está en un solo archivo, `ScrappNavHost.kt`
Vista previa	Editor de diseño de XML	Anotación `@Preview` en funciones composables	Las pantallas se revisan sin ejecutar la aplicación ni iniciar sesión

Nota. Elaboración propia con base en Android Developers (s. f.-f) y en el código del proyecto scrapp-android.

Funciones composables y recomposición

La unidad básica de Compose es la función composable, una función de Kotlin marcada con `@Composable` que emite interfaz en lugar de devolver un valor. Estas funciones se pueden anidar, reciben parámetros y admiten las estructuras de control normales del lenguaje, como `if` o `when`. Por ejemplo, la pantalla de inicio de sesión de Scrapp muestra el recuadro de error solo `if (estadoUi is AuthEstadoUi.Error)`, y no hace falta ocultar ni mostrar ninguna vista existente.

La recomposición es volver a llamar a las funciones composables cuando cambian sus entradas (Android Developers, s. f.-f). Compose no redibuja toda la pantalla, sino solo las funciones que leen el valor que cambió. Esto nos obligó a cuidar dos cosas. Una función composable tiene que ser rápida y no debe tener efectos colaterales, porque puede ejecutarse muchas veces. Y lo que deba pasar una sola vez o en segundo plano, como navegar o consultar Firebase, no puede ir en el cuerpo de la función: va en un efecto controlado o en el ViewModel.

Estado, remember y elevación de estado

En Compose, el estado es cualquier valor que puede cambiar con el tiempo y que la interfaz muestra. `mutableStateOf` crea un `MutableState<T>` observable que está integrado con el entorno de ejecución de Compose, así que cualquier cambio en su valor programa la recomposición de las funciones que lo leen (Android Developers, s. f.-e). Para que el valor no se pierda en cada recomposición se envuelve en `remember`, que lo guarda en la composición inicial y lo devuelve en las siguientes (Android Developers, s. f.-e).

El concepto que más pesó en el diseño de Scrapp fue la elevación de estado (*state hoisting*), que consiste en mover el estado al llamador de una función composable para que esta quede sin estado (Android Developers, s. f.-e). Una función sin estado solo recibe valores y avisa de los eventos mediante lambdas; se puede reutilizar y previsualizar sin problemas. Lo aplicamos en dos niveles. Los componentes de `vista/componentes/` no guardan estado propio (`ScrappTextField` recibe `value` y `onValueChange`). Las pantallas más complejas, por su parte, se dividen en una función con estado, que obtiene el ViewModel, y otra sin estado que solo dibuja. Así, `MonitorLotesScreen` delega en `MonitorLotesContent`, que recibe un `LoteUiState` y una lambda por cada acción.

El estado que pertenece a la lógica de negocio va en el ViewModel. La guía de arquitectura de Android recomienda exponerlo en un contenedor observable como `StateFlow`: la interfaz reacciona a los cambios sin pedir los datos a mano y la última versión del estado se conserva si cambia la configuración del dispositivo (Android Developers, s. f.-g). `StateFlow` es un flujo de Kotlin que siempre tiene un valor actual y emite cada actualización a quien lo observe (JetBrains, s. f.). En la vista lo convertimos en estado de Compose con `collectAsState()`.

Modifier, diseño y listas diferidas

Los modificadores (`Modifier`) ajustan el tamaño, el relleno, el fondo, el desplazamiento o la respuesta a gestos de un elemento. Se encadenan y el orden cuenta: en el chip del semáforo ponemos `.background(...)` antes de `.padding(...)` para que el color cubra también el relleno. La disposición se arma con `Column`, `Row` y `Box`, y con `Scaffold`, que deja el espacio para la barra superior, la barra inferior, el botón flotante y los mensajes emergentes.

Para colecciones de tamaño variable están las listas diferidas. `LazyColumn` y `LazyRow` solo componen y acomodan los elementos visibles, mientras que un `Column` común compone todos aunque no se vean, y eso puede afectar el rendimiento (Android Developers, s. f.-a). La documentación también recomienda dar a cada elemento una clave estable y única para que su estado se mantenga cuando cambia el orden de los datos (Android Developers, s. f.-a). En Scrapp esto importa porque el monitor ordena los lotes por urgencia, y ese orden cambia cada vez que alguien registra una merma.

Efectos secundarios

Como una función composable puede ejecutarse muchas veces, Compose tiene APIs para lanzar efectos de forma controlada. `LaunchedEffect` inicia una corrutina cuando entra en la composición y la cancela cuando sale o cuando cambian sus claves; sirve para trabajo que llama funciones `suspend`. `DisposableEffect` es para efectos que hay que limpiar cuando cambian las claves o el elemento sale de la composición (Android Developers, s. f.-d), por ejemplo registrar un observador del ciclo de vida y luego retirarlo. En Scrapp usamos `LaunchedEffect` para navegar cuando cambia el estado de autenticación y para mostrar mensajes con `Snackbar`. `DisposableEffect` lo usamos para recalcular el semáforo, la proyección y las cifras de gerencia cuando el usuario regresa a una pantalla.

Material Design 3 y temas

Material Design 3 es el sistema de diseño de Google. En Compose se aplica con el composable `MaterialTheme`, que recibe un esquema de colores, una tipografía y un conjunto de formas (Android Developers, s. f.-b), y del que toman sus colores todos los componentes de Material 3 (botones, campos de texto, tarjetas, barras de navegación). Desde Android 12 existe el color dinámico, que adapta la paleta al fondo de pantalla del usuario; si no se usa, la aplicación define sus propios esquemas claro y oscuro (Android Developers, s. f.-b).

Navigation Compose

La navegación se apoya en tres piezas. El `NavHost` es el contenedor que muestra el destino actual. El `NavController` coordina la navegación y se encarga de la pila de retroceso y de los enlaces profundos. El grafo de navegación define los destinos y cómo se conectan (Android Developers, s. f.-c). En una aplicación hecha completamente con Compose, cada destino del grafo es una función composable (Android Developers, s. f.-c). Cada destino se identifica con una ruta, que también puede llevar argumentos, como el identificador de un producto.

Arquitectura de Scrapp con Jetpack Compose

De MVC con layouts XML a MVC con funciones composables

En la Etapa 1 elegimos MVC: la vista no hace cálculos ni consultas, el controlador valida y coordina, y el modelo no sabe nada de la interfaz (CTRL+CAFE, 2026). Esa separación se mantiene en Android; lo que cambió es la comunicación entre capas. Con el sistema de vistas era común que el controlador llamara a la vista para actualizarla. En Compose eso no se puede hacer, porque la vista es el resultado de una función y no un objeto que se modifica. Por eso

implementamos los controladores como clases `ViewModel` que exponen el estado con `StateFlow`, y las pantallas se limitan a observarlo.

Con esto queda un flujo de datos unidireccional: el estado baja del ViewModel a la interfaz y los eventos suben de la interfaz al ViewModel (Android Developers, s. f.-g). Según la guía de arquitectura, este patrón da una sola fuente de verdad, permite probar el estado sin la interfaz y facilita el mantenimiento (Android Developers, s. f.-g). En nuestro caso se notó sobre todo lo de las pruebas: los motores de la Etapa 2 y sus pruebas unitarias pasaron sin cambios, porque el modelo sigue sin conocer la interfaz. La Tabla 2 resume cómo se trasladó cada capa y la Figura 1 muestra el flujo en el inicio de sesión.

Tabla 2

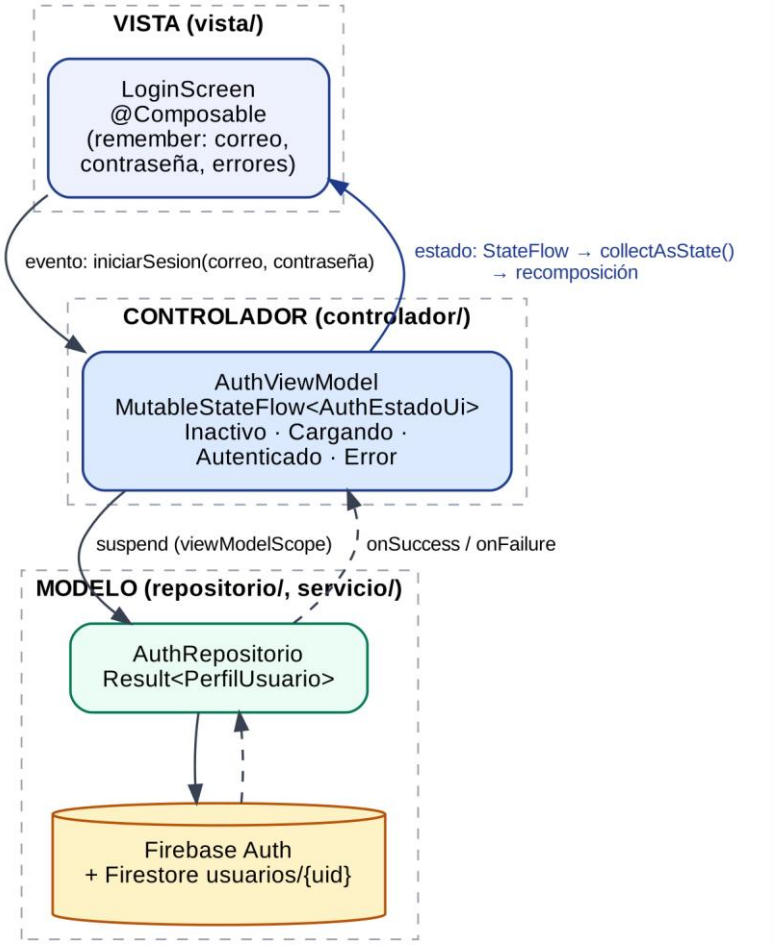
Correspondencia de las capas MVC entre la Etapa 2 y la versión Android

Capa	Etapa 2 (consola)	Etapa 3, Avance 1 (Android)	Paquete
Modelo	Entidades, repositorios en memoria y motores de cálculo	Se copiaron sin cambios; se agregó `PerfilUsuario` y `AuthRepositorio` sobre Firebase	`modelo/`, `repositorio/`, `servicio/`
Controlador	Clases que leían la consola y llamaban a la vista	Un `ViewModel` por módulo que valida, llama a los motores y expone `StateFlow`	`controlador/`
Vista	Menús numerados impresos en consola	Funciones `@Composable` que observan el estado y envían eventos	`vista/`
Utilidades	`Validador` de lectura de consola y `Logger` a archivo	`Validador` con funciones puras que devuelven el mensaje de error; `Logger` a Logcat y a `filesDir`	`util/`

Nota. Elaboración propia a partir de la estructura del repositorio scrapp-android.

Figura 1

Flujo de datos unidireccional en el inicio de sesión de Scrapp



Nota. Las flechas continuas representan eventos y llamadas; las discontinuas, resultados. El estado vuelve a la vista como `StateFlow` y provoca la recomposición de `LoginScreen`. Elaboración propia.

El estado de cada pantalla lo modelamos con tipos cerrados de Kotlin. `AuthViewModel` expone un `AuthEstadoUi` que solo puede ser `Inactivo`, `Cargando`, `Autenticado` o `Error`, y `MermaViewModel` hace lo mismo con `MermaEstadoUi`. Al ser una interfaz sellada, la vista

tiene que contemplar todos los casos en el `when`, y no se dan estados contradictorios como el indicador de carga girando junto a un mensaje de éxito.

Capa de datos compartida

En la versión de consola, `Main.kt` creaba una sola vez los repositorios y los motores y se los pasaba a cada controlador. En Android eso lo hace `ContenedorDatos`, un objeto que crea los repositorios, carga los datos iniciales e instancia los motores. Los ViewModels reciben esas instancias como parámetros con valor por defecto, así que en las pruebas se pueden reemplazar. Los módulos de Producción y Lotes las obtienen por medio de `ProveedorDependenciasModuloPrincipal`, que no crea nada propio: devuelve los mismos repositorios, la misma `ConfiguracionMotores` y los mismos `MotorProyeccion` y `MotorSemaforo` de `ContenedorDatos`. Que todos los módulos trabajen sobre los mismos datos es lo que permite la actualización dinámica. Al registrar una merma se descuenta del lote, y al volver al monitor, al dashboard o al panel de gerencia la pantalla ya muestra el cambio sin reiniciar la aplicación. En este avance los datos de negocio siguen en memoria; pasarlos a Firestore toca en el Avance 2.

Dependencias del proyecto

Usamos la lista de materiales (BOM) de Compose, que fija versiones compatibles entre todas las bibliotecas de Compose y nos ahorra coordinar versiones una por una. En la Tabla 3 están las dependencias que tienen que ver con la interfaz.

Tabla 3

Dependencias de interfaz y arquitectura del proyecto scrapp-android

Biblioteca	Versión	Uso en Scrapp
Compose BOM	2025.08.00	Fija las versiones de `ui`, `material3`, `ui-tooling` y de las bibliotecas de prueba de Compose

Biblioteca	Versión	Uso en Scrapp
Material 3 y `material-icons-extended`	Según BOM	Componentes visuales e íconos del semáforo, la barra inferior y los formularios
Navigation Compose	2.9.3	Grafo de navegación, barra inferior y paso de argumentos
Lifecycle ViewModel Compose y Runtime Compose	2.9.2	Obtener el ViewModel desde una pantalla con `viewModel()` y observar el ciclo de vida
Firebase BOM (Auth y Firestore)	34.2.0	Inicio de sesión, registro y perfil con rol en `usuarios/{uid}`
Kotlin y complemento del compilador de Compose	2.2.10	Compilación de las funciones `@Composable`

Nota. Versiones tomadas de `gradle/libs.versions.toml`. SDK mínimo 26 (Android 8.0) y SDK objetivo 36.

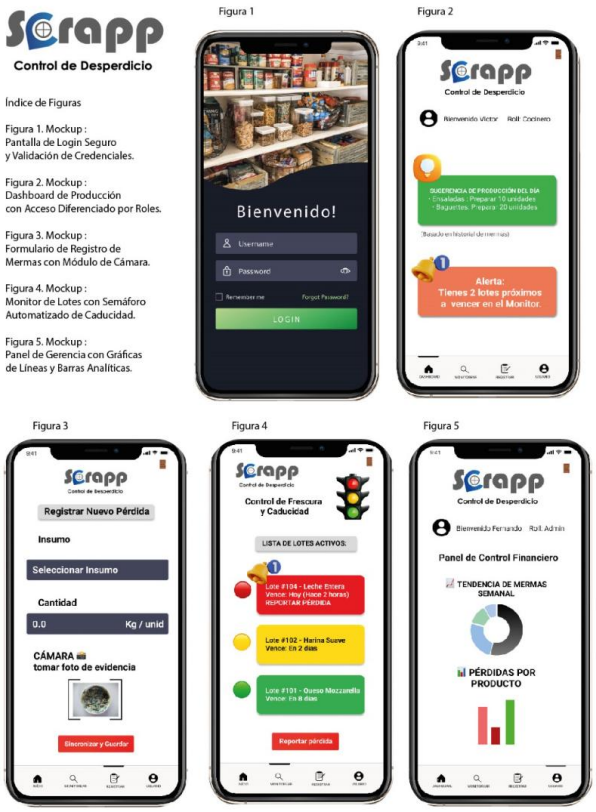
Sistema de diseño de Scrapp

Tema Material 3 a partir del prototipo

En la Etapa 1 hicimos en Figma un prototipo navegable con las cinco pantallas principales (Figura 2). Para no repetir colores en cada pantalla, definimos la paleta una sola vez en `ui/theme/Color.kt`: azul primario `#1E3A8A` para la marca y las barras, azul `#3B82F6` para acciones secundarias, fondo gris claro `#F9FAFB` y superficies blancas. Los colores del semáforo (verde `#10B981`, ámbar `#F59E0B` y rojo `#EF4444`) van aparte, porque indican el estado de un lote y no son solo decoración.

Figura 2

Prototipo de las pantallas principales de Scrapp elaborado en Figma durante la Etapa 1



Nota. Pantallas de inicio de sesión, dashboard de producción, registro de mermas, monitor de lotes y panel de gerencia. Tomado de CTRL+CAFE (2026).

`ScrappTheme` arma un `lightColorScheme` y un `darkColorScheme` con esa paleta y se los pasa a `MaterialTheme`. Dejamos el color dinámico de Android 12 desactivado por defecto (`dynamicColor = false`). Si la aplicación tomara los colores del fondo de pantalla de cada teléfono, el azul de la marca cambiaría según el dispositivo y la app dejaría de parecerse al prototipo, que es parte de lo que se evalúa en este avance. Como todos los componentes leen sus colores de `MaterialTheme.colorScheme`, el modo oscuro funciona sin tocar las pantallas.

Componentes reutilizables

Antes de que cada integrante empezara su pantalla, acordamos las firmas de ocho componentes en `vista/componentes/`. Queríamos que las pantallas se vieran iguales y que nadie tuviera que reescribir un campo de texto o una barra superior por su cuenta. La Tabla 4 resume cada componente y dónde se usa.

Tabla 4

Componentes composables reutilizables de Scrapp

Componente	Responsabilidad	Usado en
`TopBarScrapp`	Barra superior centrada con título, subtítulo opcional, botón de regreso y acciones	Dashboard, Monitor de lotes, Ingreso de lote, Panel de gerencia
`ScrappTextField`	Campo de texto con borde de marca, mensaje de error y texto de apoyo bajo el campo	Formularios de lote y merma
`BotonPrimario`	Botón principal de 48 dp de alto, con ícono opcional	Formularios
`ChipSemaforo`	Etiqueta de color para el estado de un lote o el nivel de riesgo	Monitor de lotes
`TarjetaProducto`	Tarjeta con datos resumidos de un producto	Listados de productos
`EstadoVacio`	Mensaje centrado con ícono cuando no hay datos o hubo un error	Dashboard y Monitor de lotes
`DialogoConfirmacion`	Diálogo de confirmación con acciones de aceptar y cancelar	Cierre de sesión
`BotonCerrarSesion`	Ícono de la barra superior que pide confirmación antes de salir	Dashboard y Panel de gerencia

Nota. Elaboración propia a partir del paquete `vista/componentes/` del proyecto.

`ChipSemaforo` sirve para ver por qué una función composable funciona bien como pieza reutilizable. Kotlin permite sobrecargar funciones, y por eso hay dos versiones con el mismo nombre: una recibe un texto con el nivel de riesgo y la otra recibe directamente el `EstadoCaducidad` del modelo, que es una clase sellada (Figura 3). Con la segunda, el compilador exige cubrir los cuatro estados del semáforo; si algún día se agrega un quinto estado,

la aplicación no compila hasta asignarle un color. Las dos versiones llaman a un `ChipBase` privado, así que el aspecto del chip se define en un solo lugar.

Figura 3

Sobrecarga de ChipSemaforo para el estado de caducidad de un lote

```
@Composable
fun ChipSemaforo(estado: EstadoCaducidad, modifier: Modifier = Modifier) {
    when (estado) {
        is EstadoCaducidad.Vigente -> ChipBase(
            "Verde", AlertaBaja.copy(alpha = 0.15f), AlertaBaja,
            modifier, Icons.Default.CheckCircle)
        is EstadoCaducidad.ProximoAVencer -> ChipBase(
            "Amarillo", AlertaMedia.copy(alpha = 0.15f), AlertaMedia,
            modifier, Icons.Default.Schedule)
        is EstadoCaducidad.Critico -> ChipBase(
            "Rojo", AlertaCritica.copy(alpha = 0.15f), AlertaCritica,
            modifier, Icons.Default.ReportProblem)
        is EstadoCaducidad.Vencido -> ChipBase(
            "Vencido", TextoPrincipal, Color.White,
            modifier, Icons.Default.Error)
    }
}
```

Nota. Fragmento de `vista/componentes/ChipSemaforo.kt`. Además del color, cada estado lleva un ícono distinto, para que el semáforo se distinga también sin depender del color.

Jetpack Compose aplicado a cada pantalla

Inicio de sesión y registro

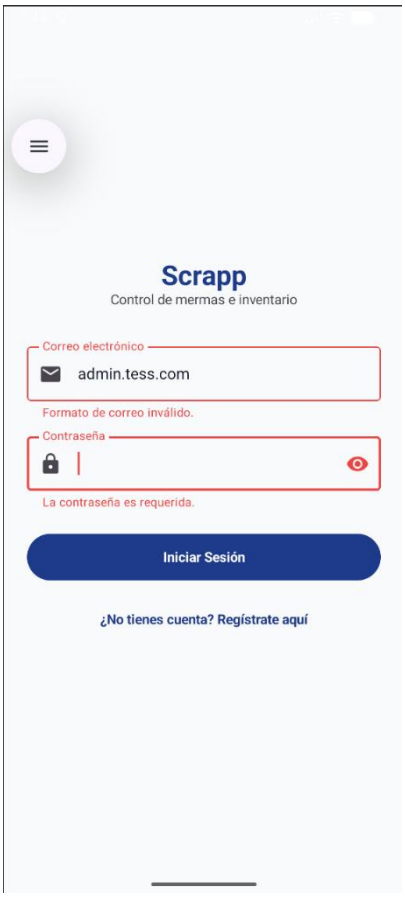
La pantalla de inicio de sesión cubre el primer requerimiento del avance. Usa `OutlinedTextField` de Material 3 para el correo y la contraseña, un `Button` para entrar y un `TextButton` para ir al registro (Figura 4). Lo que el usuario va escribiendo es estado efímero de la interfaz, así que lo guardamos con `remember { mutableStateOf("") }` dentro de la misma pantalla; mandarlo al ViewModel en cada pulsación no tendría sentido. El resultado de autenticar, en cambio, es estado de negocio y vive en `AuthViewModel`.

El botón para mostrar u ocultar la contraseña solo cambia un `Boolean`, y con eso el campo alterna entre `PasswordVisualTransformation()` y

`VisualTransformation.None`. Mientras Firebase responde, el botón se deshabilita y en lugar del texto aparece un `CircularProgressIndicator`, para que no se envíe la solicitud dos veces. Si el servidor devuelve un error (credenciales incorrectas, sin conexión o usuario desactivado), llega como `AuthEstadoUi.Error` con un mensaje en español y se muestra sobre una superficie con el color `errorContainer` del tema.

Figura 4

Pantalla de inicio de sesión con validaciones activas

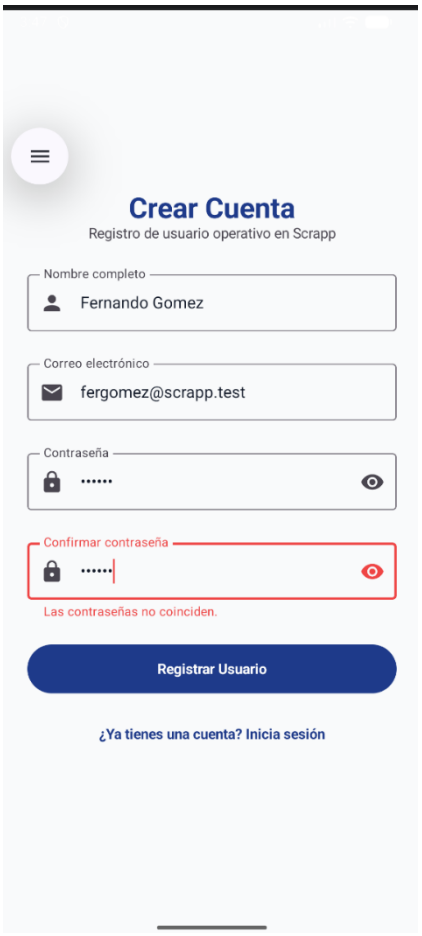


Nota. Captura de la aplicación en ejecución. Elaboración propia.

Para navegar después del inicio de sesión usamos `LaunchedEffect(estadoUi)`: cuando el estado pasa a `Autenticado`, el efecto llama a `onLoginExitoso`. Si navegáramos directamente en el cuerpo de la función, una recomposición podría repetir la navegación. La sesión persistente sale de esta misma estructura sin código extra. Al crearse, `AuthViewModel` revisa si Firebase conserva una sesión y, si la hay, emite `Autenticado`, con lo que la pantalla de inicio de sesión manda al usuario de una vez al dashboard. El registro sigue el mismo patrón con cuatro campos: nombre, correo, contraseña y confirmación (Figura 5). Todo usuario nuevo entra como Operativo, y solo un administrador puede cambiar roles desde Firestore.

Figura 5

Pantalla de registro de usuario



Nota. Captura de la aplicación en ejecución. Elaboración propia.

Dashboard de producción

El dashboard es la pantalla de inicio del rol Operativo y responde a la pregunta con la que empezó el proyecto: cuánto preparar para que no sobre. La proyección es para el día siguiente, y por eso cada tarjeta dice «Sugerido mañana». Para cada producto muestra la cantidad que calcula `MotorProyeccion`, y al lado la demanda base y el margen de seguridad aplicado, para que el cocinero sepa de dónde sale la cifra (Figura 6). La pantalla usa `Scaffold` con `TopBarScrapp`, cuyo subtítulo indica la fecha de la proyección, y una `LazyColumn` de tarjetas. Los productos sin historial suficiente se pueden filtrar, y si no hay datos aparece `EstadoVacio` en vez de una lista en blanco.

Proyectamos para mañana y no para hoy por la forma en que se calcula la tasa de merma. Esa tasa usa una ventana de 14 días que termina el día anterior a la fecha proyectada. Si el dashboard proyectara para hoy, la ventana terminaría ayer y las mermas registradas hoy no contarían, así que la cifra no cambiaría aunque el cocinero acabara de reportar una pérdida. Con la fecha de mañana, la ventana termina hoy, que es lo mismo que suponen `MotorProyeccion` y `OrquestadorMerma`. Como la pantalla se recalcula sola cada vez que el usuario vuelve a ella, una merma nueva se nota en cuanto regresa al dashboard.

Figura 6

Dashboard de producción del rol Operativo



Nota. Captura de la aplicación en ejecución. Elaboración propia.

Monitor de lotes

El monitor de lotes es la cara visible del motor del semáforo de la Etapa 2. Cada lote activo aparece en una `ElevatedCard` con el nombre del producto, su `ChipSemaforo`, las horas que le quedan y la fecha de caducidad (Figura 7). La lista es una `LazyColumn` ordenada por urgencia con `key = { it.lote.id }`, como recomienda la documentación (Android Developers, s. f.-a). Así, cuando un lote cambia de posición después de registrar una merma, Compose sabe que es el mismo elemento y no lo reconstruye.

Encima de la lista hay una fila de `FilterChip` con cuántos lotes hay en cada estado (Todos, Verde, Amarillo, Rojo y Vencido). La fila va dentro de un `Row` con desplazamiento horizontal para que quepa en pantallas angostas. El filtro elegido forma parte de `LoteUiState` y la lista filtrada se calcula como una propiedad derivada de ese estado, de modo que la vista no filtra nada por su cuenta. Un botón flotante extendido abre el ingreso de lotes y, en los lotes vencidos, la acción «Registrar merma» abre el formulario de mermas con el producto y ese mismo lote ya elegidos.

Figura 7



Monitor de lotes con el semáforo de caducidad

Nota. Captura de la aplicación en ejecución. Elaboración propia.

Para que el semáforo se actualice al volver de otra pantalla usamos `DisposableEffect` con un observador del ciclo de vida (Figura 8). Cada vez que la pantalla llega a `ON_RESUME`, el ViewModel recalcula el semáforo, y cuando la pantalla sale de la composición, `onDispose` quita el observador para no dejar referencias colgadas. La documentación oficial propone este mismo patrón para efectos que necesitan limpieza (Android Developers, s. f.-d). El dashboard y el panel de gerencia usan el mismo patrón para recalcular la proyección y las cifras financieras.

Figura 8

Recálculo del semáforo cada vez que el monitor vuelve a primer plano

```
val lifecycleOwner = LocalLifecycleOwner.current

DisposableEffect(lifecycleOwner) {
    val observer = LifecycleEventObserver { _, event ->
        if (event == Lifecycle.Event.ON_RESUME) {
            viewModel.recalcularSemaforo()
        }
    }
    lifecycleOwner.lifecycle.addObserver(observer)
    onDispose {
        lifecycleOwner.lifecycle.removeObserver(observer)
    }
}
```

Nota. Fragmento de `vista/lotes/MonitorLotesScreen.kt`.

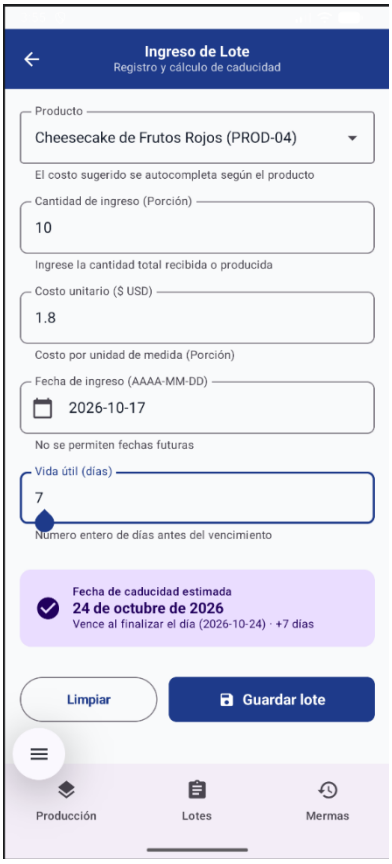
Ingreso de lotes

En el ingreso de lotes llevamos la elevación de estado más lejos que en ninguna otra pantalla. El formulario completo, con el texto de cada campo y sus errores, vive en un `IngresoLoteFormState` dentro de `LoteViewModel`, y la función `IngresoLoteContent` no guarda estado propio. Lo hicimos así porque este formulario también calcula: cuando cambian la fecha de ingreso o la vida útil, el ViewModel recalcula la fecha de caducidad y la pantalla la muestra en una tarjeta antes de guardar (Figura 9). El producto se elige con `ExposedDropdownMenuBox`, el menú desplegable de Material 3, y al seleccionarlo se sugiere

su costo unitario. Cuando el lote se guarda, un `LaunchedEffect` detecta la bandera `guardadoExitoso` y muestra un `Snackbar` con el identificador del lote nuevo.

Figura 9

Formulario de ingreso de lote con cálculo previo de caducidad



Nota. Captura de la aplicación en ejecución. Elaboración propia.

Registro de mermas

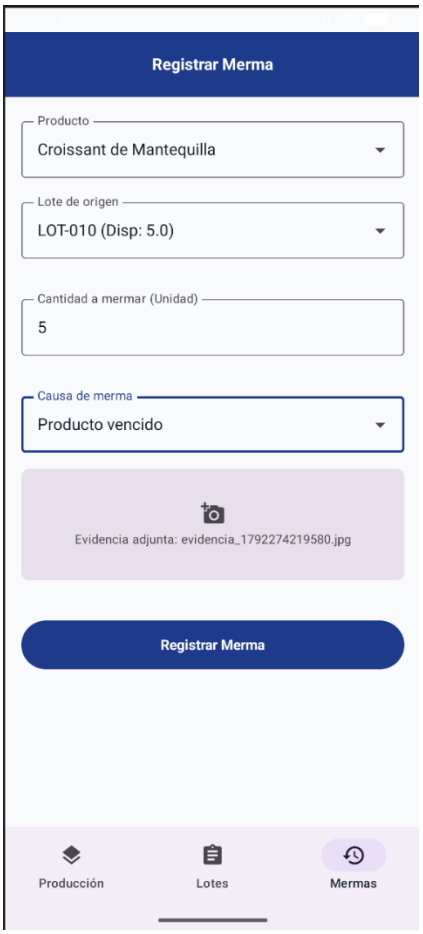
El registro de mermas es el formulario más importante del proyecto. El usuario elige el producto, el lote de donde sale la pérdida, la causa (una de las cinco del enumerado `CausaMerma`) y la cantidad, y puede adjuntar evidencia (Figura 10). Los tres selectores son `ExposedDropdownMenuBox`; en el de lotes aparece la existencia disponible y la fecha de vencimiento de cada uno, porque el usuario necesita ese dato para no registrar más de lo que hay.

Si la pantalla se abre desde el monitor de lotes, recibe el producto y el lote por la ruta `registro_merma/{productoId}/{loteId}`, y dos `LaunchedEffect` los dejan preseleccionados. El lote se preselecciona una sola vez, para no pisar la elección si el usuario decide cambiarlo.

Al guardar, `MermaViewModel` repite en la capa de lógica las validaciones que ya hizo la vista: que el lote exista, que la cantidad sea positiva y que no pase de la existencia. Luego descuenta el lote, congela el costo unitario de ese momento y llama al `OrquestadorMerma` de la Etapa 2, que recalcula el semáforo, las pérdidas y la proyección. Si algo falla, el error aparece en un `Snackbar`. Después de guardar, el botón «Registrar otra merma» vuelve a cargar los lotes del producto, para que no aparezca en la lista un lote que se quedó sin existencias. Por ahora la evidencia fotográfica se simula con un nombre de archivo que se genera al tocar el área de evidencia; la captura con la cámara usando CameraX queda para el Avance 2.

Figura 10

Formulario de registro de mermas



Nota. Captura de la aplicación en ejecución. Elaboración propia.

Panel de gerencia

El panel de gerencia solo lo ve el Administrador y trabaja con los datos reales de `MotorFinanciero`. Arriba muestra en tarjetas la pérdida del día, la pérdida del mes (del día 1 hasta hoy) y el índice de merma del mes. Debajo están los cinco productos con mayor pérdida en el mes, cada uno con una barra proporcional a su parte del total (Figura 11). Si en el mes todavía no hay mermas, aparece el mensaje «Aún no hay mermas registradas este mes.». `GerenciaViewModel` vuelve a pedir las cifras cada vez que el panel llega a `ON_RESUME`, así

que una merma recién registrada ya cuenta cuando el administrador regresa. Cada barra son dos `Box` superpuestos, y el de adentro usa `Modifier.fillMaxWidth(porcentaje)`. No hace falta ninguna biblioteca de gráficos y alcanza para comparar productos. Las gráficas de tendencia del prototipo las vamos a hacer en el Avance 2 con una biblioteca especializada.

Figura 11

Panel de gerencia del rol Administrador



Nota. Captura de la aplicación en ejecución. Elaboración propia.

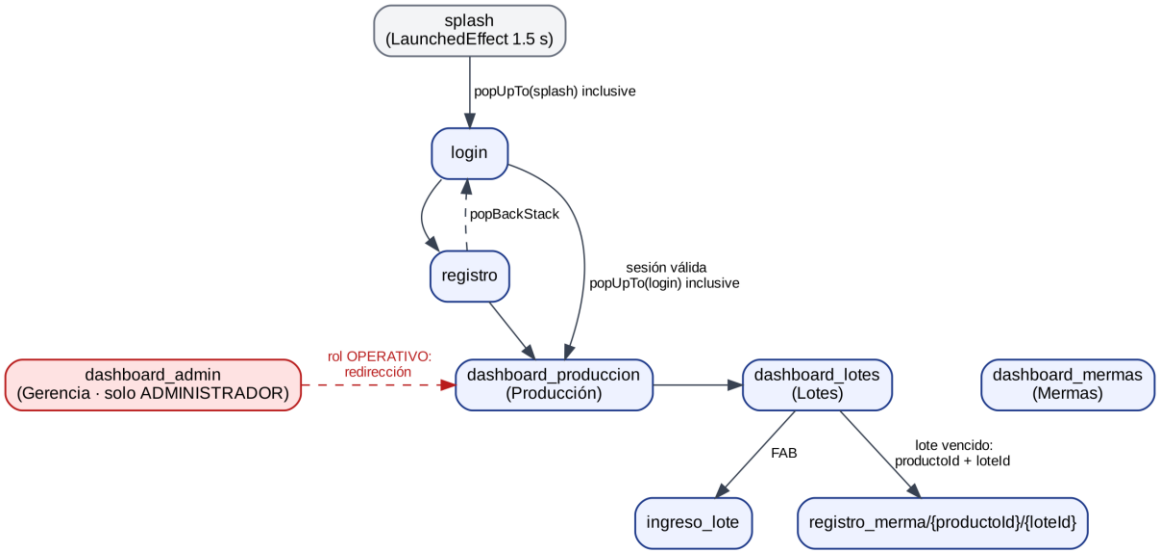
Navegación

Grafo de navegación

Toda la navegación está en `ScrappNavHost.kt`. Las rutas se definen en `Rutas.kt` como objetos de dos clases selladas: `Rutas`, para las pantallas que no están en la barra inferior, y `RutasBottom`, que además de la ruta guarda el título y el ícono de cada pestaña. Tener las rutas en clases selladas hace que un nombre mal escrito se detecte al compilar y no cuando la app ya está corriendo. La Figura 12 muestra el grafo y la Tabla 5 resume los destinos.

Figura 12

Grafo de navegación implementado en ScrappNavHost



Nota. Los cuatro destinos de la fila central son las pestañas de la barra inferior; se cambia entre ellos desde la barra, y Producción enlaza también al monitor desde su barra superior. En rojo, la redirección que impide a un usuario Operativo abrir el panel de gerencia. Elaboración propia.

Tabla 5

Destinos del grafo de navegación de Scrapp

Ruta	Pantalla	Barra inferior	Comportamiento de la pila de retroceso
`splash`	Pantalla de bienvenida	No	Se elimina de la pila al pasar al inicio de sesión
`login`	`LoginScreen`	No	Se elimina al autenticarse; «atrás» no regresa al inicio de sesión
`registro`	`RegistroScreen`	No	Vuelve al inicio de sesión con `popBackStack()`
`dashboard_produccion`	`DashboardScreen`	Sí	Raíz de las pestañas
`dashboard_lotes`	`MonitorLotesScreen`	Sí	Conserva y restaura su estado al cambiar de pestaña
`dashboard_mermas`	`RegistroMermaScreen`	Sí	Conserva y restaura su estado al cambiar de pestaña
`dashboard_admin`	`PanelGerenciaScreen`	Solo administrador	Redirige a Producción si el rol no es Administrador
`ingreso_lote`	`IngresoLoteScreen`	Sí	Regresa al monitor con `popBackStack()`
`registro_merma/{productoId}/{loteId}`	`RegistroMermaScreen`	Sí	Recibe el producto y el lote como argumentos de tipo `String`

Nota. Elaboración propia a partir de `vista/navegacion/`.

Barra inferior por rol y protección de rutas

`ScrappNavContainer` envuelve el `NavHost` en un `Scaffold` y decide si se muestra la barra inferior según la ruta actual, que obtiene con `currentBackStackEntryAsState()`.

Como es estado observable, la barra aparece y desaparece sola al navegar, y no se muestra en el inicio de sesión, el registro ni la bienvenida. `BarraInferior` arma sus elementos según el rol leído de Firestore. El Operativo ve Producción, Lotes y Mermas; el Administrador ve además Gerencia. Al cambiar de pestaña se hace `popUpTo` hacia Producción con `saveState`, `launchSingleTop` y `restoreState`. Así, tocar varias veces las pestañas no apila pantallas, y cada una recuerda su desplazamiento y sus filtros.

Esconder la pestaña no protege la pantalla, porque una ruta se puede abrir por otro camino. Por eso el destino de gerencia vuelve a revisar el rol antes de dibujar el panel (Figura 13). Si el usuario no está autorizado, la redirección se hace dentro de un `LaunchedEffect`. Como dice el comentario en el código, navegar durante la composición provoca recomposiciones en bucle, y es un buen ejemplo práctico de por qué los efectos tienen que ir separados del dibujo. Al cerrar sesión se vacía toda la pila con `popUpTo(navController.graph.id) { inclusive = true }`, para que el botón «atrás» no lleve de vuelta a pantallas protegidas.

Figura 13

Protección del panel de gerencia según el rol del usuario

```
composable(RutasBottom.Gerencia.route) {
    val esAdmin = usuarioActual?.esAdministrador == true
    if (esAdmin) {
        PanelGerenciaScreen(
            navController = navController,
            onCerrarSesion = cerrarSesion
        )
    } else {
        // Navegar durante la composición provoca recomposiciones en bucle
        LaunchedEffect(Unit) {
            navController.navigate(RutasBottom.Produccion.route) {
                popUpTo(RutasBottom.Gerencia.route) { inclusive = true }
            }
        }
    }
}
```

Nota. Fragmento de `vista/navegacion/ScrappNavHost.kt`. La protección de la interfaz se complementa con las reglas de Firestore, que impiden que un usuario cambie su propio rol.

Validación de formularios

Las reglas de validación están en `util/Validador.kt` como funciones puras: reciben el texto del campo y devuelven el mensaje de error, o `null` si el valor está bien. Esto se conecta directo con dos parámetros de los campos de Material 3: `isError = error != null` pinta el campo de rojo y `supportingText` pone el mensaje debajo. Como no dependen de la interfaz, estas funciones se pueden probar con pruebas unitarias comunes.

Decidimos validar la primera vez al pulsar el botón de envío y, desde ahí, volver a validar cada campo mientras el usuario escribe. Marcar errores desde la primera letra en un formulario vacío nos pareció agresivo. En cambio, cuando el usuario ya sabe qué está mal, ver que el error desaparece al corregirlo le confirma que lo hizo bien. La Tabla 6 resume las validaciones de cada formulario.

Tabla 6

Validaciones implementadas en los formularios de Scrapp

Formulario	Campo	Regla
Inicio de sesión	Correo	Obligatorio y con formato válido (`Patterns.EMAIL_ADDRESS`)
Inicio de sesión	Contraseña	Obligatoria y de al menos 6 caracteres
Registro	Nombre	Obligatorio
Registro	Correo y contraseña	Mismas reglas que en el inicio de sesión
Registro	Confirmación	Obligatoria e igual a la contraseña
Ingreso de lote	Producto	Debe seleccionarse uno del catálogo
Ingreso de lote	Cantidad y costo unitario	Numéricos y mayores que cero
Ingreso de lote	Fecha de ingreso	Formato AAAA-MM-DD y no posterior a la fecha actual
Ingreso de lote	Vida útil	Número entero de días, al menos 1
Registro de merma	Lote	Debe seleccionarse uno con existencia disponible
Registro de merma	Cantidad	Numérica, mayor que cero y no mayor que la existencia del lote

Nota. Los mensajes de Firebase (credenciales incorrectas, correo ya registrado o sin conexión) se traducen al español en `AuthRepositorio` y se muestran por separado de estas validaciones. Elaboración propia.

Análisis propio: ventajas y limitaciones

Ventajas observadas durante el desarrollo

Lo primero que notamos fue cuánto código nos ahorramos. El monitor de lotes tiene filtros, conteos, ordenamiento, estado vacío, indicador de carga y tarjetas, y aun así cabe en una sola función que se lee de arriba abajo en el mismo orden en que aparece en pantalla. Con el sistema de vistas habríamos necesitado un layout XML, un `Adapter`, un `ViewHolder` y código para alternar la visibilidad entre la lista, el indicador de carga y el mensaje vacío. En Compose esos tres casos son ramas de un `when`.

Con Compose, fallas típicas de sincronización, como un indicador de carga que no deja de girar o un mensaje de error que no se quita, se volvieron difíciles de cometer, porque la pantalla no guarda nada que pueda quedar desfasado y simplemente muestra lo que dice el estado. Haber modelado ese estado con interfaces selladas ayudó todavía más, porque el compilador avisa si falta algún caso.

Compose también nos facilitó trabajar en paralelo. Como acordamos el tema y los componentes reutilizables al principio, y cada pantalla es una función con parámetros y lambdas, cada uno pudo desarrollar y previsualizar su módulo con datos de prueba antes de que existieran la navegación o la capa de datos. Al final el grafo de navegación las conectó sin modificar las pantallas, porque ninguna, salvo el panel de gerencia, depende del `NavController`.

Además, pasar a Compose no nos obligó a reescribir la lógica de la Etapa 2. Copiamos los motores y sus pruebas tal cual. Para nosotros fue la prueba de que separar el modelo de la vista en la Etapa 1 valió la pena: el cambio de tecnología solo tocó la capa de vista.

Limitaciones y mejoras para el Avance 2

La integración nos dejó una lección. Como cada módulo se desarrolló por separado, Producción y Lotes terminaron con sus propios repositorios en memoria, distintos de los que usaba `MermaViewModel`. La aplicación compilaba y cada pantalla funcionaba por su cuenta, pero registrar una merma no cambiaba el monitor ni el dashboard, porque cada módulo miraba una copia diferente de los datos. Lo corregimos haciendo que `ProveedorDependenciasModuloPrincipal` delegue en `ContenedorDatos`, y agregamos la prueba `IntegracionMermaTest`, que revisa que los módulos compartan las mismas instancias y que una merma de hoy cambie el lote, la proyección de mañana y la pérdida del día. El problema no estaba en Compose: las pantallas se recomponían bien, solo que cada una con datos distintos.

Quedan varias cosas por mejorar. Las pantallas observan el estado con `collectAsState()`, pero la guía de arquitectura recomienda `collectAsStateWithLifecycle()`, que deja de recolectar cuando la aplicación pasa a segundo plano (Android Developers, s. f.-g). La dependencia ya está en el proyecto, así que el cambio es sencillo.

Las rutas, por otro lado, son texto dentro de clases selladas. La versión actual de Navigation Compose acepta rutas de cualquier tipo serializable (Android Developers, s. f.-c), con lo que podríamos pasar el producto y el lote como propiedades tipadas en vez de armar la cadena `registro_merma/{productoId}/{loteId}`. `PanelGerenciaScreen` recibe el

`NavController` como parámetro, mientras que las demás pantallas exponen lambdas; conviene igualarla al resto para poder previsualizarla y probarla por separado.

En la integración también quedaron validaciones duplicadas. `LoteViewModel` tiene sus propias funciones porque `Validador` todavía no estaba fusionado cuando se desarrolló ese módulo, y lo correcto es que delegue en `Validador` para que las reglas y los mensajes no terminen siendo distintos. Por último, la aplicación aún no tiene pruebas de interfaz. La biblioteca `ui-test-junit4` ya está en el proyecto, y los formularios de inicio de sesión y registro serían un buen punto de partida.

Conclusiones

Jetpack Compose le quedó bien a Scrapp porque la aplicación consiste, sobre todo, en mostrar estados que cambian: un lote que pasa de verde a rojo, una merma que baja una existencia, una sesión que se abre o se cierra. Describir la pantalla como función del estado se ajusta mejor a eso que actualizar vistas a mano.

La arquitectura MVC de la Etapa 1 se mantuvo con un solo ajuste: los controladores ahora son ViewModels que exponen el estado con `StateFlow`, y la vista lo observa en lugar de recibir órdenes. De ahí salió un flujo de datos unidireccional que nos permitió trabajar en paralelo, reutilizar sin cambios la lógica de la Etapa 2 y evitar errores de sincronización entre los datos y la interfaz.

Cada decisión de diseño importante se apoyó en algo concreto de Compose. La elevación de estado nos dio componentes reutilizables y el monitor de lotes usa una lista diferida con claves estables. Con los efectos secundarios separamos la navegación y el recálculo del dibujo de la pantalla, y Material 3 con un tema propio mantuvo la app parecida al prototipo. Las limitaciones que encontramos (recolección de estado sin tomar en cuenta el ciclo de vida, rutas

basadas en texto y validaciones duplicadas) tienen solución conocida y están en el plan del Avance 2, junto con pasar los datos a Firestore y capturar la evidencia con la cámara.

Referencias

Android Developers. (s. f.-a). *Lazy lists and lazy grids*. Google. Recuperado el 17 de octubre de 2026, de https://developer.android.com/develop/ui/compose/lists

Android Developers. (s. f.-b). *Material Design 3 in Compose*. Google. Recuperado el 17 de octubre de 2026, de https://developer.android.com/develop/ui/compose/designsystems/material3

Android Developers. (s. f.-c). *Navigation with Compose*. Google. Recuperado el 17 de octubre de 2026, de https://developer.android.com/develop/ui/compose/navigation

Android Developers. (s. f.-d). *Side-effects in Compose*. Google. Recuperado el 17 de octubre de 2026, de https://developer.android.com/develop/ui/compose/side-effects

Android Developers. (s. f.-e). *State and Jetpack Compose*. Google. Recuperado el 17 de octubre de 2026, de https://developer.android.com/develop/ui/compose/state

Android Developers. (s. f.-f). *Thinking in Compose*. Google. Recuperado el 17 de octubre de 2026, de https://developer.android.com/develop/ui/compose/mental-model

Android Developers. (s. f.-g). *UI layer*. Google. Recuperado el 17 de octubre de 2026, de https://developer.android.com/topic/architecture/ui-layer

Bellini, A.-C. y Butcher, N. (2021, 28 de julio). Jetpack Compose is now 1.0: Announcing Android's modern toolkit for building native UI. *Android Developers Blog*. https://android-developers.googleblog.com/2021/07/jetpack-compose-announcement.html

CTRL+CAFE. (2026). *Proyecto de cátedra. Etapa 1: Análisis y planificación del proyecto Scrapp* [Trabajo no publicado]. Dirección de Educación a Distancia, Universidad Don Bosco.

JetBrains. (s. f.). *StateFlow*. Kotlin coroutines API reference. Recuperado el 17 de octubre de 2026, de https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-state-flow/